

Laboratoire 3

Création d'un objet « vecteur » (allocation dynamique)

Ce laboratoire est à faire **individuellement**. Vous allez créer un nouvel objet « vecteur » qui sera une copie du **vecteur dynamique** des librairies standards. Il pourra ainsi modifier sa dimension au lieu d'être fixe à la déclaration (comme le serait plutôt un tableau conventionnel).

Voici le contenu initial de votre nouvel objet vecteur :

```
class vecteur
{
    private:
        int* _tab;           // Pointeur sur un vecteur d'entiers
        int _dim;           // Taille du vecteur

    public:
        vecteur();           // Constructeur sans paramètre.
                             // Attention, un pointeur comme *_tab ne
                             // peut pas rester non initialisé.

        vecteur(int dim);    // Initialise le vecteur avec une taille
                             // donnée par le paramètre d'entrée dim.

        ~vecteur();          // Destructeur essentiel avec l'allocation
                             // dynamique. Attention, un pointeur comme
                             // *_tab ne peut pas conserver l'adresse d'un
                             // espace mémoire désalloué.
};
```

Voici les premiers tests à réaliser dans la fonction main() :

```
int main()
{
    vecteur v1;              // Appel du constructeur sans paramètre
    vecteur v2(5);           // Appel du constructeur avec une taille de 5
    vecteur v3(0);           // Appel du constructeur avec une taille nulle
                             // (construction d'un vecteur vide)

    return 0;
}
```

Consignes générales

- Utilisez la fonction `assert()` pour prévenir tout éventuel problème avec l'allocation en mémoire (les cas limite). Par exemple, on ne peut pas allouer `[0]` ou `[-1]` avec l'opérateur `new[]`. Le nombre doit être strictement supérieur à 0 lorsque vient le temps d'utiliser l'opérateur `new[]`.
- Privilégiez la notation `*_tab` au lieu de `_tab[0]` pour atteindre les éléments de votre vecteur. Le seul endroit où les crochets devraient apparaître dans les méthodes de votre classe est lors de l'allocation avec l'opérateur `new[]` et lors de la

désallocation avec l'opérateur **delete[]**. Autrement, aucun crochet ne devrait être utilisé.

- Prenez soin de laisser dans la fonction main() toutes les instructions pour tester les possibilités de votre vecteur. **Pour vous aider, reproduisez les tests du jeu d'essais fourni à la fin de ce document** (tous les tests s'y trouvent et sont dans le bon ordre).

Étape 1 – Constructeurs et destructeur

Construisez la classe « vecteur » avec la déclaration montrée ci-dessus et programmez les constructeurs et le destructeur avec l'allocation dynamique. Testez vos méthodes avec les points d'arrêts.

Étape 2 – Copieur

Codez le constructeur de copie (aussi appelé copieur) pour effectuer l'opération suivante avec votre vecteur :

```
vecteur v4(v1), // v4 est une copie de v1 et est donc vide
          v5(v2); // v5 est une copie de v2 et a une taille de 5
```

** Rappelez-vous que le copieur peut être appelé à votre insu si vous passer un objet par valeur en paramètre d'entrée d'une fonction ou méthode ou si vous retourner un objet par valeur à la sortie d'une telle fonction ou méthode.*

Étape 3 – Méthode print() et surcharge de l'opérateur <<

À cette étape, vous êtes prêt à coder les méthodes permettant d'afficher le contenu du vecteur. Ce faisant, codez la méthode print(ostream& output) et la surcharge de l'opérateur << :

Méthode	Description
print(ostream& output)	Affiche les nombres contenus dans le vecteur les uns après les autres, séparés par un espace.
Surcharge de l'opérateur <<	Appelle la méthode print(ostream& output).

Étape 4 – Méthode push_back()

Programmez la méthode push_back(int nb) qui permet d'ajouter une case mémoire à la fin du vecteur et d'insérer le nombre nb dans cette nouvelle case mémoire. Pour ce faire, il faut :

- Créer un autre vecteur avec une taille dim+1.
- Copier tout le contenu de l'ancien vecteur vers le nouveau vecteur et ce, jusqu'à atteindre la dernière case mémoire venant d'être ajoutée.
- Insérer le nombre nb dans la dernière case mémoire du nouveau vecteur.
- Supprimer l'ancien vecteur (désallocation).

 **Question de réflexion :** Notre push_back est peu efficace car il doit recopier le tableau au complet chaque fois qu'on l'appelle. Qu'est-ce qu'on pourrait faire pour éviter cela? (Ne changez pas votre code, nous y reviendrons en classe)

Étape 5 – Méthode size()

Programmez la méthode size(), laquelle doit retourner la dimension (propriété _dim) du vecteur. À titre d'exemple, lorsqu'on construit un vecteur de la sorte : **vecteur v1(10);** et qu'on exécute ensuite l'instruction **v1.size();** on s'attend à avoir le nombre 10 comme taille pour v1.

Étape 6 – Méthode resize()

Programmez la méthode resize(int dim) qui reçoit la nouvelle taille du vecteur en paramètre d'entrée et qui redimensionne le vecteur à cette taille. **Attention, plusieurs cas sont à prévoir :**

- Aucun changement ne doit être fait si la nouvelle taille du vecteur correspond à l'ancienne.
- Une nouvelle taille négative doit être interceptée par un assert() (il faut faire volontairement planter le programme dans ce cas).
- Une taille trop importante (dépassant la mémoire disponible) doit être interceptée par un try/catch().
- Il faut pouvoir donner une dimension non nulle à un vecteur initialement vide.
- Il faut pouvoir rendre vide un vecteur dont la nouvelle taille doit être nulle.
- Il faut pouvoir rapetisser un vecteur initialement plus grand. Dans ce cas, on doit conserver le contenu des cases mémoires occupant un rang inférieur à la nouvelle taille du vecteur.
- Il faut pouvoir augmenter la taille d'un vecteur initialement plus petit. Dans ce cas, on doit conserver le contenu de toutes les cases mémoire de l'ancien vecteur et ajouter de nouvelles cases mémoire (dont le contenu n'est pas initialisé) à la fin.

** Plusieurs tests sont à prévoir pour la méthode resize(). Référez-vous au jeu d'essais à la fin de ce document pour les implémenter.*

Étape 7 – Méthode clear()

Codez la méthode clear() qui permet d'effacer le contenu du vecteur en mémoire et de désallouer l'adresse où il se trouvait. La taille du vecteur (propriété _dim) doit aussi être remise à la fin de la méthode clear().

** Une fois votre méthode clear() fonctionnelle, vous pouvez vous contenter de l'appeler dans le destructeur du vecteur.*

Étape 8 – Méthode at() et surcharge de l'opérateur []

Programmez la méthode at(int index) et la surcharge de l'opérateur [] pour permettre aux utilisateurs d'accéder directement à un des éléments du vecteur. Pour éviter de répéter du code, la surcharge de l'opérateur [] pourra appeler la méthode at().

La méthode at(int index) prend la position d'un élément du vecteur en paramètre d'entrée et **retourne une référence de l'élément** à cette position. La surcharge de l'opérateur [] fait de même (elle prend un index en paramètre d'entrée et retourne une référence en sortie).

** Un assert() doit faire planter le programme si la position de l'élément ne se retrouve pas entre les limites du vecteur, soit entre 0 et la taille (exclue) du vecteur.*

Par exemple, on pourrait faire dans le main() :

```
vecteur v5(v2);

for (int i = 0; i < v5.size(); i++){
    v5[i] = i;
    v5.at(i) = i * 2 + 1;
    cout << v5[i] << v5.at(i);
}
```

Étape 9 - Affectateur

Programmez l'affectateur (surcharge de l'opérateur =) pour effectuer les opérations suivantes avec votre vecteur :

```
v2 = v4;          // v2 (vide) est maintenant identique à v4 (non vide)
v2 = v1;          // v2 (non vide) est maintenant identique à v1 (vide)
v3 = v2;          // v3 (vide) est maintenant identique à v2 (vide)
v3 = v4 = v5;     // v3(vide) et v4 (non vide) sont identiques à v5 (non vide)
```

Aucun traitement ne devrait être entrepris si on tente d'Affecter un vecteur à lui-même, par exemple :

```
v5 = v5; // L'affectateur ne devrait rien faire dans ce cas-ci et le
          // vecteur v5 devrait rester exactement comme il était avant.
```

Étape 10 – Surcharge de l'opérateur ==

Pour effectuer les opérations suivantes avec votre vecteur, codez la surcharge de l'opérateur de comparaison (opérateur ==) :

```
if (v2 == v4)
    cout << "v2 et v4 sont identiques";

else
    cout << "v2 et v4 ne sont pas identiques";

// Vous pouvez utiliser l'opérateur ternaire pour effectuer l'affichage
// et la comparaison en une seule ligne de code (moins lisible par contre)
cout << "v2 et v4" << ((v2 == v4) ? " sont " : " ne sont pas ") << "identiques";
```

Pour l'opérateur de comparaison, vous devez d'abord vérifier si les dimensions des deux vecteurs sont égales. Si elles ne le sont pas, on retourne « false », mais si elles le sont, il faut alors vérifier si leur contenu, élément par élément, est identique. **La valeur « true » n'est retournée que si les tailles et les contenus sont identiques entre les deux vecteurs.**

Étape 11 – Surcharge de l'opérateur +

La concaténation de deux vecteurs comme suit : $v3 = v1 + v2$; crée un nouveau vecteur (**attention, ce nouveau vecteur n'est pas v3**) dont la dimension est égale à la somme des dimensions de v1 et de v2 et dont le contenu revient à celui de v1 suivi de celui de v2.

Pour offrir une telle fonctionnalité, surchargez l'opérateur +. Ayez en tête les cas particuliers où un utilisateur ferait $v1 = v1 + v2$; ou encore $v4 = v1 + v2 + v3$; Autrement dit, il faut retourner un tout nouveau vecteur à l'issue de l'opérateur et permettre également les opérations chaînées (plusieurs concaténations en une seule ligne de code).

Étape 12 – Surcharge de l'opérateur +=

Vous voulez permettre aux utilisateurs de votre classe de concaténer 2 vecteurs ensemble tout en l'affectant dans le vecteur implicite comme ceci `v3 += v1`, ce qui ajoute `v1` à `v3`.

Étape 13 – Fonction swap()

La méthode `swap()` devrait permettre d'échanger le contenu entre deux vecteurs :

```
v1.swap(v3); // Le contenu de v1 se retrouve dans v3 et vice-versa
v3.swap(v1); // Même opération que la précédente ligne de code
```

Or, qu'un utilisateur écrive `v1.swap(v3)` ou `v3.swap(v1)`, le résultat devrait être le même. Pour rendre plus explicite l'utilisation de la méthode `swap()`, il serait intéressant d'en faire une fonction (plutôt qu'une méthode) pouvant être appelée comme suit :

```
swap(v1, v3); // swap(v3, v1); est aussi possible et mène au même résultat
```

Pour que la fonction `swap()` ait accès aux membres privés du vecteur (`*_tab` et `_dim`), il faudra la rendre amie (« friend ») avec la classe. Programmer cette fonction amie `swap()` et faites attention de ne pas recréer deux vecteurs supplémentaires en mémoire : on souhaite simplement échanger les adresses dans ce cas-ci (plus efficace).

Étape 14 – Méthodes reverse() et reverseRecursive() (versions itérative et récursive)

Codez les méthode `reverse()` (méthode itérative) et `reverseRecursive()` (méthode récursive) dans la classe « vecteur ». La méthode `reverseRecursive()` doit bien sûr s'appeler elle-même pour être considérée comme récursive. Vous n'aurez donc pas besoin de boucle dans son implémentation contrairement à la méthode `recursive()`.

```
v1.reverse();
cout << "Le contenu de v1 inversé itérativement est : " << v1;

v1.reverseRecursive();
cout << "Le contenu de v1 inversé récursivement est : " << v1;
```

Étape 15 – Patron de classe (« template »)

Transformez votre classe « vecteur » en patron (« template ») pour pouvoir déclarer des vecteurs de n'importe quel type primitif ou type d'objets comme ceci : `vecteur<int>`, `vecteur<char>`, `vecteur<string>`, etc.

Pour que les « templates » fonctionnent, vous devez faire toutes les modifications suivantes avant de pouvoir compiler. **Faites une copie de votre vecteur d'entiers sur votre ordinateur avant d'entreprendre les modifications suivantes pour au moins conserver une version fonctionnelle.**

- En C++, les patrons de classe fonctionnent si et seulement si les méthodes sont codées **avec la définition de la classe dans le même fichier**. En effet, une précompilation est nécessaire pour traduire les patrons de classe en un type connu (`int`, `char`, `string`, etc.) et cette précompilation impose que les occurrences à remplacer soient toutes réunies dans le même fichier. Ainsi, copiez-collez les méthodes de votre fichier `.cpp` vers votre fichier `.h` en-dessous de la définition de la classe « vecteur ». Retirez ensuite le fichier `.cpp` du projet. **Après cette étape, compiler pour être sûr que cette modification n'a pas créé d'erreurs.**

- Ajoutez l’instruction du « template » au-dessus de la classe « vecteur », au-dessus des méthodes et partout où le pointeur `*_tab` ou encadre le type de donnée « vecteur » sont utilisés comme le montre d’ailleurs les exemples suivants :

```
template <class TYPE>
class vecteur {
    TYPE* _tab; // Le mot TYPE sera utilisé pour remplacer int
    int _dim;   // et généraliser votre vecteur en TYPE. En
                // revanche, _dim doit rester en int.
};
// Constructeur
template <class TYPE> // On ajoute ceci au-dessus de chaque méthode
vecteur<TYPE>::vecteur() {
    _dim = 0;
    _tab = nullptr;
}

// Méthode resize()
template <class TYPE>
void vecteur<TYPE>::resize(int newSize) {
    assert(newSize >= 0);

    TYPE* newTab = nullptr;

    if (newSize > 0)
        newTab = new TYPE[newSize];
    ...
}

// Méthode at()
template <class TYPE>
TYPE& vecteur<TYPE>::at(int index) const {
    ...
}

// Méthode size()
template <class TYPE>
int vecteur<TYPE>::size() const {
    ...
}

// Affectateur
template <class TYPE>
vecteur<TYPE>& vecteur<TYPE>::operator=(const vecteur<TYPE>& vect) {
    ...
}
```

- Ajoutez aussi **dans la définition de la classe** au-dessus du prototype de la fonction « friend » `swap()` l’instruction :

```
template <class T>
friend void swap(vecteur<T>&, vecteur<T>&);
```

- Modifiez dans votre `main()` toutes les déclarations de vecteurs pour être en mesure d'avoir des vecteurs du type désiré. Par exemple :

```
vecteur<int> v1, v2, v3, v4, v5;
vecteur<char> v6;
vecteur<string> v7;
```

Étape 16 – Intégration de votre vecteur dans un jeu développé avec SFML

Une fois votre vecteur pleinement testé à l'aide du jeu d'essais ci-après (ce jeu d'essais est déjà codé dans votre `main()` : il vous suffit de décommenter les tests associés), tentez de l'intégrer au programme *Whac-A-Mole* que vous avez développé lors du laboratoire précédent.

Contentez-vous seulement de remplacer les vecteurs `_appearingRects`, `_hitRects`, `_disappearingRects` et `_disappearingHitRects` dans la classe `Mole` (fichier « `Mole.h` ») par votre propre instance de vecteur :

```
vecteur<sf::IntRect> _appearingRects;
vecteur<sf::IntRect> _hitRects;
vecteur<sf::IntRect> _disappearingRects;
vecteur<sf::IntRect> _disappearingHitRects;
```

L'intégration de votre propre vecteur dans « `Mole.h` » devrait se faire sans que vous n'ayez besoin d'apporter de modifications à votre jeu *Whac-A-Mole*. En revanche, **il vous faudra ajouter cette méthode dans le fichier de votre propre vecteur pour permettre l'utilisation de listes d'initialisation** à l'instanciation des vecteurs `_hitRects`, `_disappearingRects` et `_disappearingHitRects` :

```
#include <initializer_list>

template <class TYPE>
vecteur<TYPE>::vecteur(const std::initializer_list<TYPE>& list) {
    _dim = list.size();

    if (_dim == 0)
        _tab = nullptr;

    else {
        int i = 0;

        _tab = new TYPE[_dim];

        for (const TYPE& item : list) {
            *(_tab + i) = item;
            i++;
        }
    }
}
```

Jeu d'essais pour le vecteur

** Un fichier contenant la fonction main() avec tous les tests listés dans le tableau ci-dessous est fourni avec cet énoncé de laboratoire. Il vous suffit de décommenter les tests des méthodes au fur et à mesure que vous les implémentez pour vérifier leur conformité d'exécution.*

Description du test	Test	Résultat attendu
1. constructeur sans paramètre avec paramètre	vecteur v1;	Vecteur vide (nullptr,)
	vecteur v2(5);	Vecteur de 5 int (&tab, 5)
	vecteur v3(0);	Vecteur vide (nullptr, 0)
	vecteur erreur(-1);	Erreur de assert
2. copieur	vecteur v3(v1)	Vecteur vide (nullptr, 0)
	vecteur v5(v2)	Vecteur de 5 int (&tab, 5)
3. print et opérateur<<	v1.print(cout);	N'affiche rien car vide
	v2.print(cout);	Affiche les 5 nombres (0 ou non init : -842150451)
	cout << v3;	N'affiche rien car vide
	cout << v5;	Affiche 5 nombres et 5 au bout
4. push back	v4.push_back(5)	Ajout de 5 à un vecteur vide
	v5.push_back(5)	Ajout de 5 à un vecteur non vide
5. size	cout << v1.size();	Affiche 0
	cout << v2.size();	Affiche 5
6. resize		
	- vecteur vide agrandi à 3	Agrandi à 3 int et affecte à 0 ou à -842150451
	- vecteur non vide réduit à 0	devient vide (nullptr,0)
	- vecteur de 5 réduit à 2	réduit à 2
	- vecteur de 2 resized à 2	Reste tel quel et ne fait pas de copie
	- vecteur non vide agrandi à 6	Agrandi de 2 à 6 int et ajoute 4 fois 0 ou -842150451
	- erreur de paramètre	Assert nous averti
7. clear	v1.clear();	Devient vide (nullptr,0)
	v2.clear();	Reste vide (nullptr,0)
8. at et operator[]	v5.at(0) = 1;	Affecte 1 à la position 0
	v5.at(1) = 3;	Affecte 3 à la position 1
	Faîtes une boucle pour initialiser v5 à 1 3 5 7 9 11	
	v1.at(0) = 5;	vecteur vide, un assert averti
	v5.at(2) = 3;	En dehors des limites, assert
	v5.at(-2) = 3;	Paramètre invalide, assert
9. affectateur	v2 = v4;	Vecteur vide = vecteur non vide, v2 et v4 = 5
	v2 = v1;	Vecteur non vide = vecteur vide, v2 devient (nullptr, 0)
	v3 = v2	2 vides, les 2 restent vides
	v5 = v5;	Vecteur avec lui-même, ne fait rien
	v3 = v4 = v5;	Vecteur v3 et v4 deviennent (1,3,5,7,9,11) dim 6 et on test le return
10. operator==	if(v2 == v4) cout << «=»; else cout << «pas =»;	2 vecteurs de taille différentes dont 1 vide, affiche pas =
	cout<<((v3 == v5)?«=»:«pas =»);	2 vecteurs de taille et contenu égal, affiche =
	v3.at(0) = 3; cout<<((v3 == v5)?«=»:«pas =»);	2 vecteurs de taille égal, mais contenu inégal, affiche pas =
	cout<<((v1 == v2)?«=»:«pas =»);	2 vecteurs vides, affiche =

11. operator+	<code>cout << v2 + v1;</code>	concaténer 2 vecteurs vides, affiche rien
	<code>cout << v2 + v3;</code>	Concaténer 1 vecteur vide avec un non vide affiche (3,3,5,7,9,11) dim 6
	<code>cout<< v2 + v3 +v5;</code>	Concaténer 2 vecteurs non vides et 1 vide (3,3,5,7,9,11,1,3, 5,7,9,11) dim 12
	<code>cout << v3 + v3</code>	Concaténer un vecteur avec lui-même, affiche (3,3,5,7,9,11,3,3, 5,7,9,11) dim 12
12. operator+=	<code>v2 += v1;</code>	Ajouter un vide à un vide, reste vide (nullptr, 0)
	<code>v2 += v3;</code>	Ajouter 1 vecteur vide à un non vide devient (3,3,5,7,9,11)
	<code>v3 += v4 + v5;</code>	Concaténer 3 vecteurs non vides devient (3,3,5,7,9,11,1,3, 5,7,9,11,1,3, 5,7,9,11) dim 18
	<code>v2 += v2</code>	Concaténer un vecteur avec lui-même, devient (3,3,5,7,9,11, 3,3,5,7,9,11) dim 12
13. swap	<code>v2.clear();</code>	Pour avoir 2 vecteurs vide
	<code>swap(v1, v2);</code>	Swap de 2 vecteurs vides, (nullptr, 0)
	<code>swap(v3, v4);</code>	Swap de 2 vecteurs non vides
	<code>swap(v2, v5);</code>	Swap d'un vecteur vide, avec non vide
14. reverse	<code>v3.reverse();</code>	Devient (11,9,7,5,3,3)
	<code>v3.reverseRecursive();</code>	redevient (3,3,5,7,9,11)
	<code>v1.reverse();</code>	inversion d'un vecteur vide, fait rien
	<code>v1.reverseRecursive();</code>	mais ne plante pas
15. template	<code>vecteur<char> v6; vecteur<string> v7;</code>	Appel de toutes les méthodes avec un vecteur de char ou de string